# Sapphire Stay




**Session: 2022-2026**

**Submitted By:**

**Ahmer Mazhar (FA22-BCS-001)**

**Anas Arshad Chaudhary (FA22-BCS-018)**

**Sharjeel Arshad (FA22-BCS-028)**

**Supervised By:**

**Ms. Rabia Rauf**

**Lecturer**

**Department of CS&IT, Mirpur University of Science and Technology, Mirpur AJK**

**A Dissertation Submitted To**

**Department of CS&IT,**

**Mirpur University of Science and Technology, Mirpur AJK**

**As a Partial Fulfillment of the Requirement for the Award of the Degree of**

**Bachelors of Science in Computer Science Technology**

## Final Approval

This is to certify that we have read the report submitted by ***Ahmer Mazhar (FA22-BCS001), Anas Arshad Chaudhary (FA22-BCS-018), and Sharjeel Arshad (FA22-BCS-028)*** for the partial fulfillment of the requirements for the degree of the Bachelors of Science in Computer Science (BSCS). It is our judgment that this report is of sufficient standard to warrant its acceptance by Mirpur University of Science and Technology for the degree of the Bachelors of Science in Computer Science (BSCS).

?>,

## Committee:

**1**

_____

Ms. Rabia Rauf

Supervisor

**2**

_____

Name of External Examiner

External Examiner

Date:

**3**

_____

Dr. Faisal Riaz

Head of Department

# Declaration

We hereby declare that this document *Sapphire Stay* neither as a whole nor as a part has been copied out from any source. It is further declared that we have developed this project along with the accompanying report entirely based on our personal efforts, under the supervision of our project advisor Mam Rabia Rauf. If any part of the system is found to be copied or reproduced from any external source, we shall be fully responsible for the consequences.

---

**Ahmer Mazhar**

**FA22-BCS-001**

---

**Anas Arshad Chaudhary**

**FA22-BCS-018**

---

**Sharjeel Arshad**

**FA22-BCS-028**

# Dedication

This project *Sapphire Stay*, is dedicated to travellers and tourists who seek a simple, reliable, and efficient way to book hotel rooms without unnecessary complications. Many visitors face difficulties in finding available rooms, unclear pricing, and lack of real-time booking systems in smaller cities like Muzaffarabad.

This platform is designed to simplify that experience by providing a smooth, digital solution for room discovery and booking. It represents a step toward modernizing local hospitality services and improving the overall user experience for both customers and hotel management.

# Acknowledgement

First of all, we are obliged to Allah Almighty the Merciful, the Beneficent and the source of all Knowledge, for granting us the courage and knowledge to complete this Project.

Secondly, we want to thank our supervisor Ms. Rabia Rauf for her continuous support throughout the project. She helped and guided us in such an easy way that the completion of our project didn't seem difficult at all.

**Ahmer Mazhar**
**FA22-BCS-001**

**Anas Arshad Chaudhary**
**FA22-BCS-018**

**Sharjeel Arshad**
**FA22-BCS-028**

# Abstract

Sapphire Stay is a web-based hotel management system developed to simplify and digitize room booking and hotel operations. It allows customers to browse rooms, check availability, and book rooms, while staff and administrators view bookings, payments, and room data efficiently. It includes key features such as booking management, payment handling and invoice generation. By using this application people can easily book rooms from anywhere.

# Table of Contents


Abstract ..... 7
Chapter 1: ..... 12
Introduction ..... 12
1.1 Background ..... 13
1.2 Goals and Objectives ..... 13
1.3 Solution Overview ..... 13
1.4 Report Outline ..... 13
Chapter 2: ..... 14
Literature / Market Survey ..... 14
2.1 Introduction ..... 15
2.2 Literature Review / Technologies Overview .....15
2.2.1 Existing System----------15
2.2.2 Limitations of Existing Systems----------15
2.2.3 Proposed System----------15
2.2.4 Importance of Hotel Management Systems----------16
2.2.5 Technological Overview----------16
2.3 Summary .....16
Chapter 3: ..... 17
Requirement Analysis ..... 17
3.1 Introduction .....18
3.1.1 Problem Scenarios .....18
3.1.2 Proposed Solution: .....18
3.2 Functional Requirements .....19
3.3 Non-Functional Requirements .....20
Chapter 4: ..... 21
System Design ..... 21
4.1 Introduction ..... 22
4.2 Use Case Diagram ..... 23
4.3 Use Case Scenarios ..... 24-27
4.4 Conceptual Database Design ..... 27
4.4.1 ERD of Sapphire Stay: ..... 27
4.5 Activity Diagram .....28-31
4.6 Physical Database Design ..... 32-37
Chapter 5: ..... 38
Implementation ..... ..... 38
5.1 Introduction.....39
5.2Pseudocode..... 39-42
Chapter 6: ..... 43
Testing ..... ..... 43
6.1 Introduction.....44
6.2 Testing ..... 44
6.2.1 Black Box Testing ..... 44

6.2.2 White Box Testing ..... 44
6.3 Black Box Testing..... 45-50
Chapter 7..... ..... 51
Conclusion and Future Enhancement ..... 51
7.1Introduction ..... 52
7.2 Results and Discussions..... ..... 52
7.3 Future Enhancements ..... 53
7.4 Summary ..... 53

# List of Tables


TABLE 4.3.1: USE CASE SCENARIO OF LOGIN FOR SAPPHIRE STAY .....22
TABLE 4.3.2: USE CASE SCENARIO OF BROWSE ROOMS FOR SAPPHIRE STAY.....23
TABLE 4.3.3: USE CASE SCENARIO OF MANAGE BOOKINGS FOR SAPPHIRE STAY.....24
TABLE 4.3.4: USE CASE SCENARIO OF EDIT ROOMS FOR SAPPHIRE STAY .....25
TABLE 4.3.5: USE CASE SCENARIO RECORD PAYMENT FOR SAPPHIRE STAY.....26
TABLE 4.6.1: PHYSICAL DATABASE DESIGN FOR USERS ..... 32
TABLE 4.6.2: PHYSICAL DATABASE DESIGN FOR ROOMS ..... 32
TABLE 4.6.3: PHYSICAL DATABASE DESIGN FOR ROOM CATEGORIES ..... 33
TABLE 4.6.4: PHYSICAL DATABASE DESIGN FOR AMENETIES ..... 33
TABLE 4.6.5: PHYSICAL DATABASE DESIGN FOR ROOM AMENETIES ..... 33
TABLE 4.6.6: PHYSICAL DATABASE DESIGN FOR ROOM IMAGES ..... ..33
TABLE 4.6.7: PHYSICAL DATABASE DESIGN FOR BOOKINGS ..... 34
TABLE 4.6.8: PHYSICAL DATABASE DESIGN FOR PAYMENTS ..... 34
TABLE 4.6.9: PHYSICAL DATABASE DESIGN FOR INVOICES ..... 35
TABLE 4.6.10: PHYSICAL DATABASE DESIGN FOR REVIEWS..... 35
TABLE 4.6.11: PHYSICAL DATABASE DESIGN FOR MESSAGES ..... 36
TABLE 4.6.12: PHYSICAL DATABASE DESIGN FOR CONTACT MESSAGES..... 36
TABLE 4.6.13: PHYSICAL DATABASE DESIGN FOR GALLERY..... 36
TABLE 4.6.14: PHYSICAL DATABASE DESIGN FOR SESSIONS ..... 37
TABLE 4.6.15: PHYSICAL DATABASE DESIGN FOR SETTINGS ..... 37
TABLE 6.3.1: BLACK BOX TESTING FOR USER REGISTRATION ..... 45
TABLE 6.3.2: BLACK BOX TESTING FOR LOGIN ..... 45
TABLE 6.3.3: BLACK BOX TESTING FOR LOGOUT ..... 46
TABLE 6.3.4: BLACK BOX TESTING FOR VIEW ROOMS ..... 46
TABLE 6.3.5: BLACK BOX TESTING FOR ROOM BOOKING ..... 47
TABLE6.3.6: BLACK BOX TESTING FOR PAYMENT PROCESSING ..... 48
TABLE 6.3.7: BLACK BOX TESTING FOR BOOKING MANAGEMENT ..... 48
TABLE 6.3.8: BLACK BOX TESTING FOR ROOM MANAGEMENT ..... 48
TABLE 6.3.9: BLACK BOX TESTING FOR CONTACT MESSAGES ..... 49
TABLE 6.3.10: BLACK BOX TESTING FOR INVOICE GENERATION ..... 50

# List Of Figures

FIG 4.2: USE CASE DIAGRAM OF SAPPHIRE STAY ..... 22
FIG 4.4.1 ERD OF SAPPHIRE STAY ..... 27
FIG 4.5.1: ACTIVITY DIAGRAM FOR LOGIN USE CASE ..... 28
FIG 4.5.2: ACTIVITY DIAGRAM FOR USER REGISTRATION ..... 29
FIG 4.5.3: ACTIVITY DIAGRAM FOR BOOKING ..... 30
FIG 4.5.4: ACTIVITY DIAGRAM FOR STAFF FLOW ..... 31

# Chapter 1

# Introduction

## 1.1 Background

Sapphire Stay, a hotel in Muzaffarabad, currently uses manual methods to manage bookings and customer records, leading to inefficiencies and record keeping problems. Customers also face delays in confirming reservations. To address this, Sapphire Stay is developed as a web-based system that allows users to view available rooms, make bookings, and with proper database manage reservations, payments, and room data efficiently.

## 1.2 Goals and Objectives

The main objectives of Sapphire Stay are to develop a simple and user-friendly room booking system, provide real-time room availability, simplify the reservation process for customers, assist hotel staff in managing bookings efficiently, reduce errors associated with manual systems, and implement the solution using modern technologies.

## 1.3 Solution Overview

Sapphire Stay is a web-based system designed to manage hotel room bookings efficiently. It allows users to browse rooms, check availability, and make reservations, while staff can manage bookings and update room data in real time. It replaces manual processes with an automated solution, improving accuracy, reliability, and overall user experience. Users can book from anywhere. Data is managed in proper database which reduces paperwork.

## 1.4 Report Outline

This report describes the development of Sapphire Stay, a web-based hotel booking system. Chapter 1 introduces the project, while Chapter 2 covers literature review. Chapter 3 defines system requirements, and Chapter 4 covers design including Use Case Diagram, activity diagram and database structure. Chapter 5 explains implementation and testing, followed by conclusions and future work.

# Chapter 2

# Literature Review

## 2.1 Introduction

This chapter provides an overview of existing hotel booking and management approach in Sapphire Stay i-e manual methods. It also discusses the general technologies used in such systems and highlights common limitations found in traditional manual and database-less approaches. Furthermore, the chapter presents the proposed solution based on the identified issues, along with relevant concepts that support the analysis and design of the system..

## 2.2 Literature Review / Technologies Overview

### 2.2.1 Existing System

Sapphire Stay currently operates using manual processes for all aspects of hotel management, including room bookings, invoicing, customer records, and overall administration. These manual methods are time-consuming and increase the chances of errors and data inaccuracy during daily operations.

### 2.2.2 Limitations of Existing Systems

Sapphire Stay manual system is inefficient and prone to errors, data loss, and duplication. Moreover there is no proper database to store data which leads to problems in monitoring large customer and hotel data. These limitations make it difficult to manage bookings, maintain accurate room availability, and ensure smooth operational flow.

### 2.2.3 Proposed System

Sapphire Stay is a web-based hotel management system developed specifically for a single hotel. It streamlines booking and administrative processes by allowing customers to view room availability and make reservations, while hotel staff can manage bookings, rooms, and user data through a centralized system. The system is designed to be simple, efficient, and tailored to the specific needs of a single hotel.

### 2.2.4 Importance of Hotel Management Systems

Hotel management systems are essential for improving efficiency and reducing manual workload. They automate booking processes, manage customer data, track room availability, and enhance service delivery. Such systems help reduce errors, improve accuracy, and provide a better overall experience for both customers and hotel staff.

### 2.2.5 Technologies Overview

Sapphire Stay is developed using widely used web technologies. PHP is used for backend development and server-side processing, while MySQL is used for database management and storing relational data. The frontend is built using HTML, CSS, Tailwind CSS, and JavaScript to create a responsive and user-friendly interface. XAMPP is used as the local server environment, and Visual Studio Code is used as the primary development tool.

## 2.3 Summary

This chapter discussed existing hotel management approaches and their limitations, highlighting the need for a dedicated system. It introduced Sapphire Stay as a tailored solution for efficient hotel operations and outlined the technologies used in its development. The next chapter will focus on requirement analysis and system design.

# Chapter 3

# Requirement Analysis

## 3.1 Introduction

This chapter identifies the functional and non-functional requirements of the Sapphire Stay system. The system is designed for a real hotel that previously managed all operations manually, including bookings, room allocation, billing, and record keeping. This chapter highlights the problems in the existing manual workflow and defines how the proposed system improves efficiency, accuracy, and management of hotel operations.

### 3.1.1 Problem Scenarios

The hotel previously relied on manual processes for all operations, which created several practical issues:

- Room bookings were recorded manually, leading to chances of double booking or human error
- Availability of rooms had to be checked physically or through registers, causing delays
- Guest records were maintained on paper, making data management difficult and unorganized
- Billing and payment records were handled manually, increasing the risk of mistakes
- Staff coordination was difficult due to lack of centralized information sharing
- Tracking past bookings and customer history was time-consuming and inefficient These limitations made hotel operations slow, less reliable, and difficult to manage, especially during high occupancy periods.

### 3.1.2 Proposed Solution

To overcome the limitations of the manual system, Sapphire Stay provides a centralized web-based hotel management system that digitizes all major operations. The system allows:

- Room availability management to prevent double booking
- Online and structured booking process for customers
- Centralized database for storing all customer, booking, and room data

- Automated booking status tracking (pending, confirmed, checked-in, checked-out, cancelled)
- Digital management of invoices and payment records
- Easy access for staff to manage hotel operations from a single dashboard

This solution reduces manual workload, improves accuracy, and ensures smooth and efficient hotel operations.

## 3.2 Functional Requirements

The Sapphire Stay system provides the following functional requirements:

- **User Registration:**
  Customers can register by providing their name, email, phone number, and password to access booking features.
- **Login System:**
  Registered users can securely log in to access their dashboard and booking features, while staff and admin access their respective panels.
- **Room Management:**
  Admin can add, update, delete, and manage rooms, categories, and room details.
- **Room Browsing:**
  Users can view available rooms with details such as price, capacity, description, images, and status.
- **Booking System:**
  Users can book rooms by selecting check-in and check-out dates along with guest details.
- **Booking Management:**
  Staff and admin can view all bookings, including confirmation, cancellation, and status updates.
- **Payment Management:**
  The system admin records and manages payments with different statuses such as pending, completed, or failed.

- **Invoice Generation:**

  Each confirmed booking automatically generates an invoice with complete payment and booking details.
- **Reviews System:**

  Customers can submit reviews and ratings for rooms after their stay.
- **Gallery Management:**

  The system manages hotel images including rooms, amenities, and exterior views.
- **Messaging/Contact System:**

  Users can send inquiries or messages to hotel administration.

## 3.3 Non-Functional Requirements

The system ensures the following quality attributes:

- **Availability:**

  The system operates 24/7 for booking and management access.
- **Usability:**

  Simple and user-friendly interface for customers and hotel staff.
- **Performance:**

  Fast response time for searching rooms and processing bookings.
- **Security:**

  The system provides secure authentication and role-based access control to ensure that only authorized users can access specific functionalities and data.

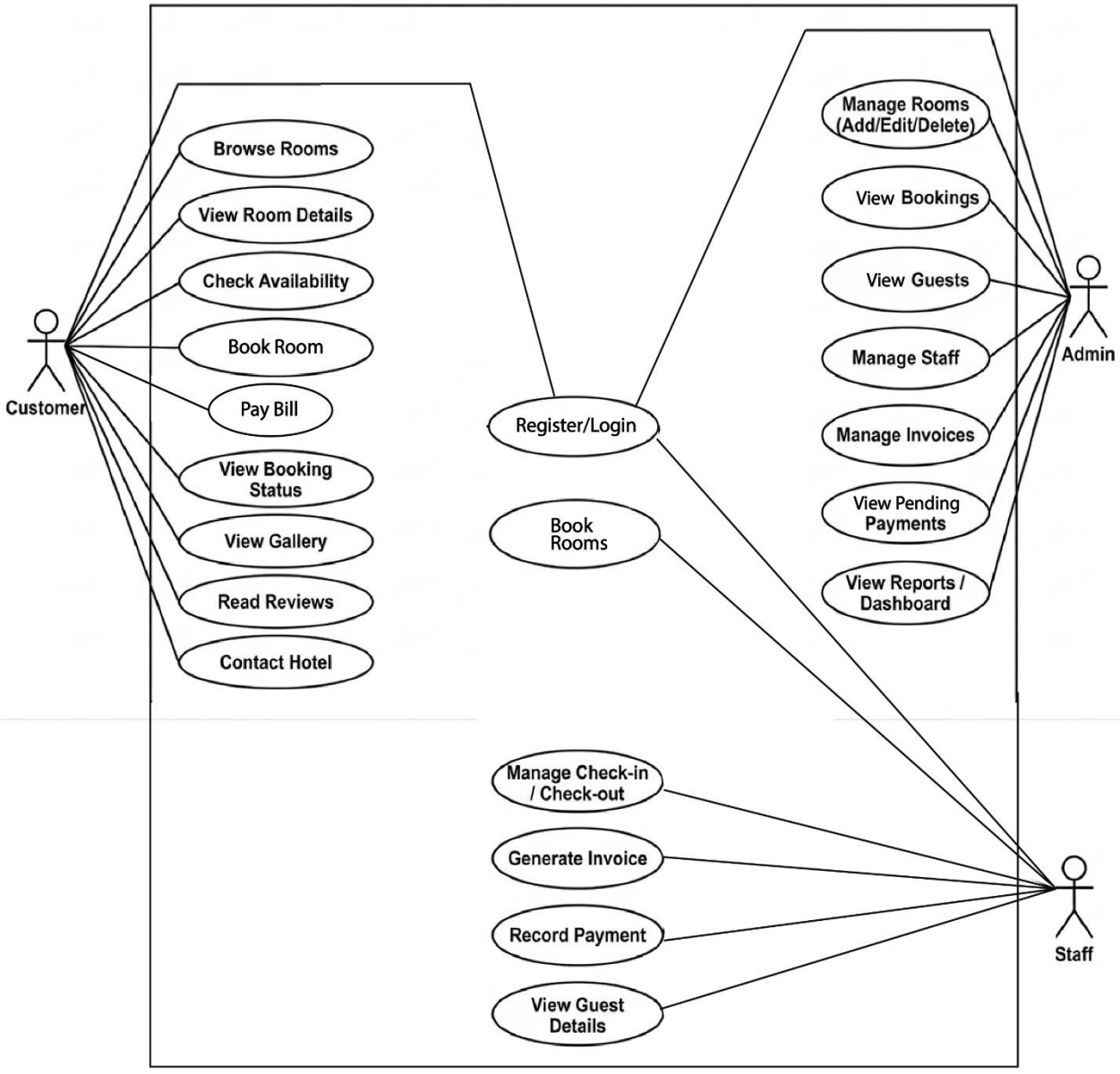
# Chapter 4

# System Design

## 4.1 Introduction

System design is a crucial stage in software development. It provides a blueprint of how the system will function and how users will interact with it. This chapter explains the user interactions, use case diagram, and overall design of the Sapphire Stay hotel management system.

## 4.2 Use Case Diagram



## 4.3 Use Case Scenarios for Sapphire Stay

**Table 4.3.1: Use case scenario of Login for Sapphire Stay**

| **Login** | |
|---|---|
| **Attribute** | **Details** |
| **Precondition** | - User opens browser on their device<br>- User must be registered in the system |
| **Postcondition** | -Registered user can access their dashboard and perform allowed actions |
| **Normal Flow** | - User opens website<br>- Enters email and password - Clicks "Login"<br>- System verifies credentials<br>- User is logged in successfully |
| **Exception** | - Invalid email or password<br>- Empty input fields<br>- Incorrect password (retry)<br>- Server or network failure<br>- Retry |

**Table 4.3.2: Use case scenario of Browse Rooms for Sapphire Stay**

| **Browse Rooms** | |
|---|---|
| **Attribute** | **Details** |
| **Precondition** | - User opens browser on their device |
| **Postcondition** | - User can view available rooms with details |
| **Normal Flow** | - User opens website<br>- Navigates to rooms section<br>- System fetches room data<br>- Rooms are displayed with details (price, type, availability) |
| | - No rooms available |
| **Exception** | - Data loading failure<br>- Server/network error |

**Table 4.3.3: Use case scenario of Manage Bookings for Sapphire Stay**

| **Manage Bookings** | |
|---|---|
| **Attribute** | **Details** |
| **Precondition** | - Staff must be logged in |
| **Postcondition** | - Booking status is updated successfully |
| **Normal Flow** | - Staff logs into system<br>- Navigates to bookings section<br>- Views list of bookings<br>- Selects a booking<br>- Updates status<br>- System saves changes |
| **Exception** | - Booking not found<br>- Invalid update action<br>- Server/database failure |

**Table 4.3.4: Use case scenario of Add/Edit/Delete Rooms for Sapphire Stay**

| **Add/Edit/Delete Rooms** | |
|---|---|
| **Attribute** | **Details** |
| **Precondition** | - Admin/Staff must be logged in |
| **Postcondition** | - Room data is updated in system |
| **Normal Flow** | - User opens room management section<br>- Adds new room or selects existing room<br>- Edits or deletes room details<br>- Saves changes<br>- System updates database |
| **Exception** | - Invalid input data<br>- Unauthorized access<br>- Server/database error |

**Table 4.3.5: Use case scenario of Record Payment for Sapphire Stay**

| **Record Payment** | |
|---|---|
| **Attribute** | **Details** |
| **Precondition** | - Staff must be logged in<br>- Booking must exist |
| **Postcondition** | - Payment is recorded successfully |
| **Normal Flow** | - Staff opens booking details<br>- Enters payment information<br>- Confirms payment<br>- System records payment |
| **Exception** | - Invalid payment details<br>- Payment already recorded<br>- Server/database failure |

## 4.4 Conceptual Database Design

In the conceptual design of the system ERD (Entity Relationship Diagram) is included. An entity relationship diagram is a data modelling technique that creates a graphical representation of all the entities and the relationships between them.

### 4.4.1 ERD of Sapphire Stay:

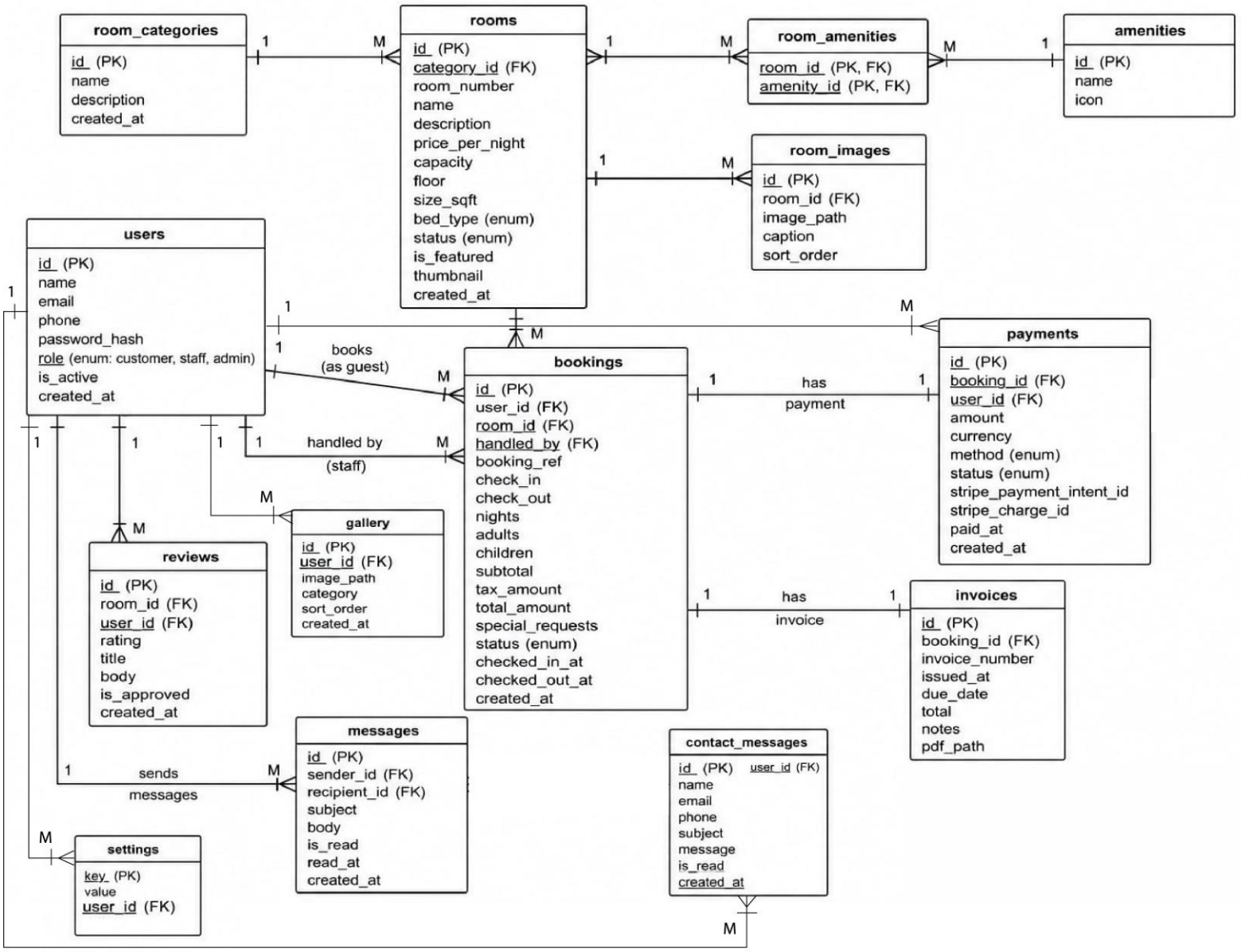



**Figure 4.4.1: ERD**

## 4.5 Activity Diagram

An Activity Diagram is a type of UML behavioural diagram that visually represents the workflow of a system or process. It uses standardized symbols to illustrate the flow of control from one activity to another, including decision points, parallel actions, and start/end states. Activity diagrams are commonly used during the design phase to understand and communicate how different processes interact and to identify potential improvements or issues within the workflow.

**Activity Diagram for Login Functionality**

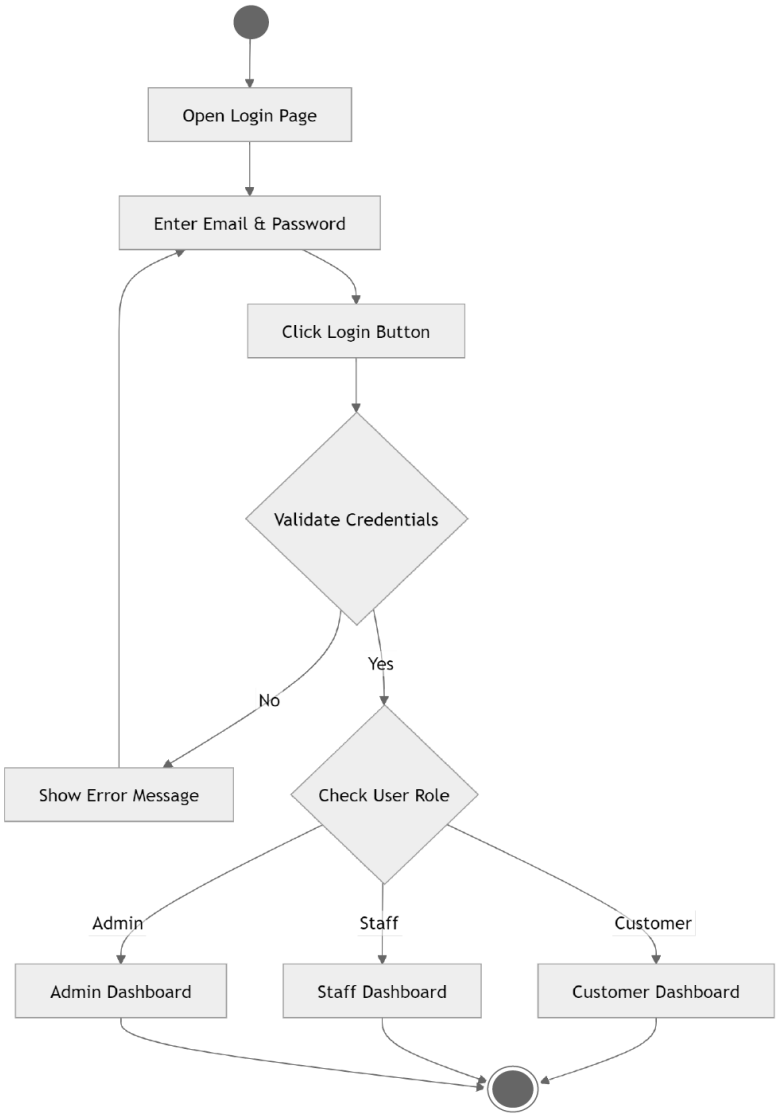



**Figure 4.5.1: Activity Diagram for Login Functionality**

## Activity Diagram for User Registration

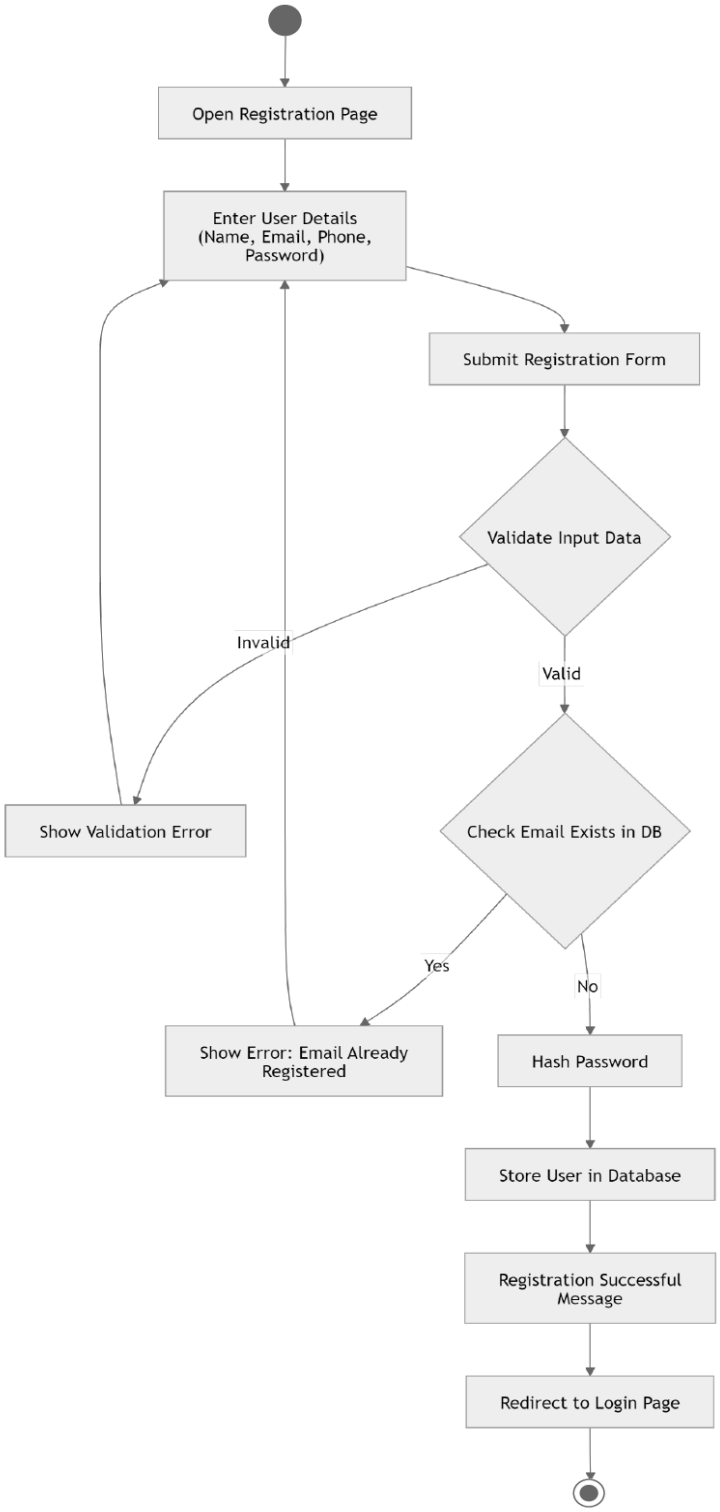



**Figure 4.5.2: Activity Diagram for User Registration**

## Activity Diagram for Booking

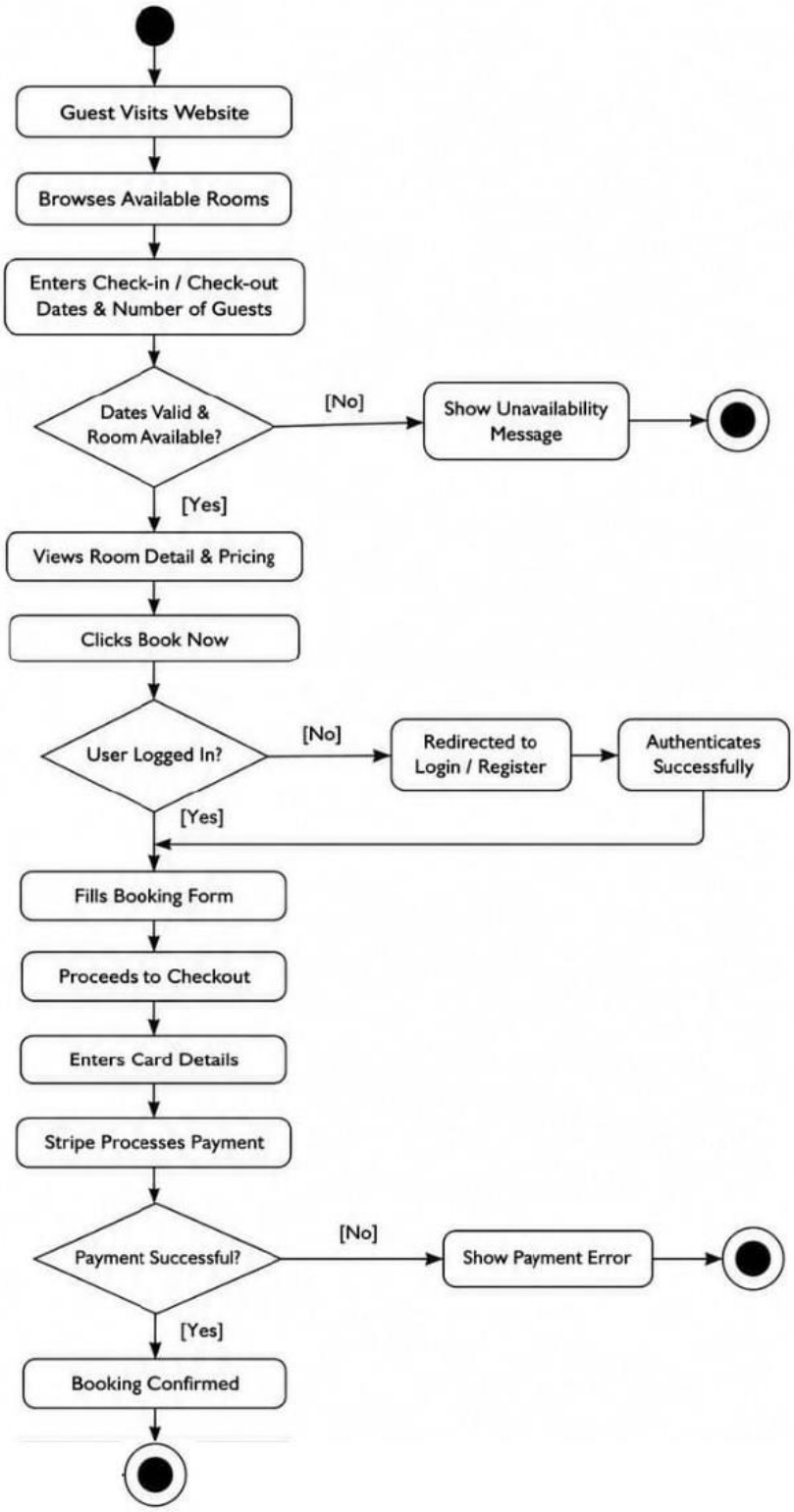



**Figure 4.5.3: Activity Diagram for Booking**

## Activity Diagram for Staff Flow

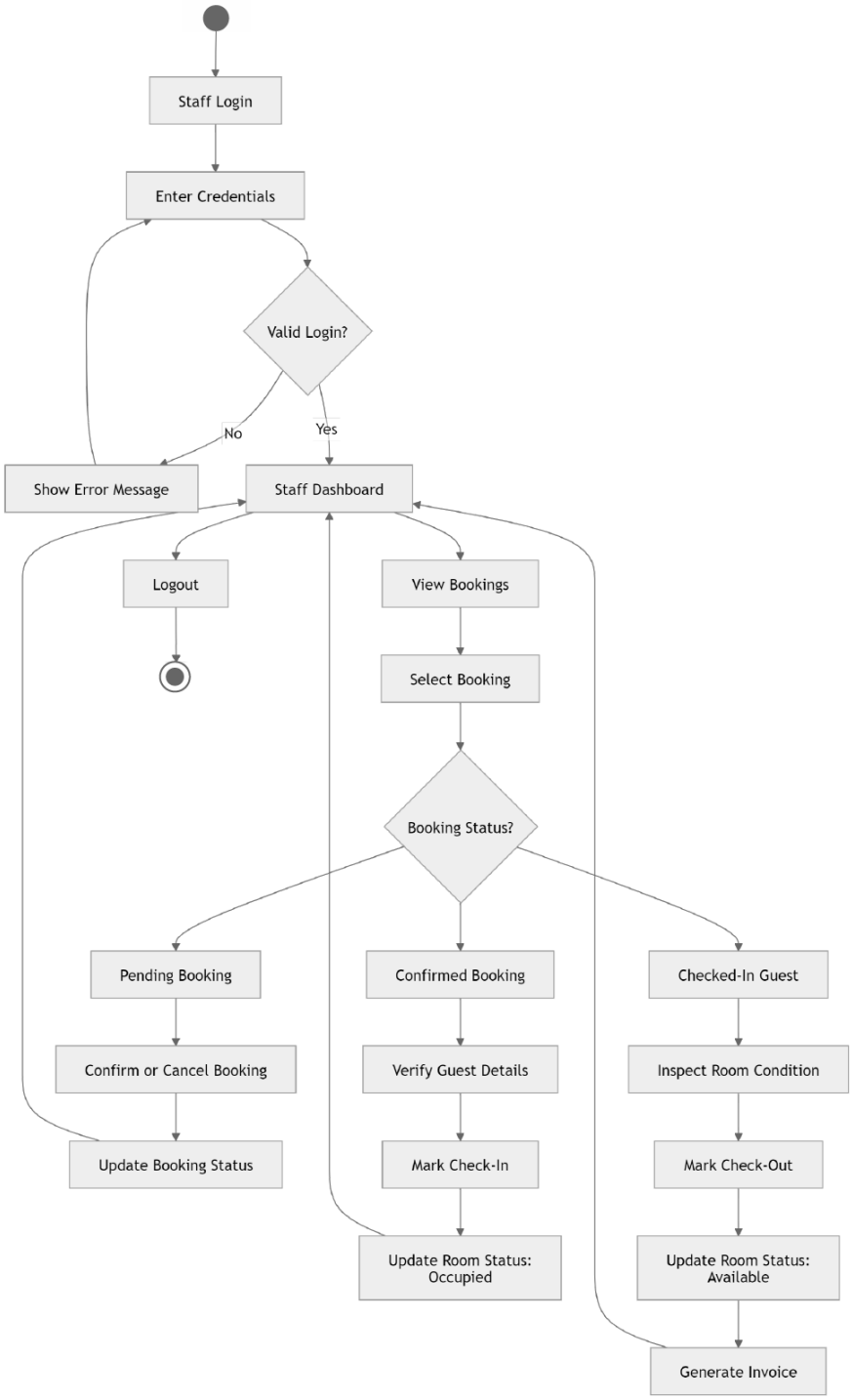



**Figure 4.5.4: Activity Diagram for Staff Flow**

## 4.6 Physical Database Design

**Table 4.6.1: Physical Database Design for Users**

### Users

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| name | varchar(120) |
| email | varchar(191) |
| phone | varchar(30) |
| password_hash | varchar(255) |
| role | enum('customer','staff','admin') |
| avatar | varchar(255) |
| is_active | boolean |
| email_verified_at | datetime |
| remember_token | varchar(100) |
| created_at | datetime |
| updated_at | datetime |

**Table 4.6.2: Physical Database Design for Rooms**

### Rooms

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| category_id | int (FK → room_categories.id) |
| room_number | varchar(20) |
| name | varchar(120) |
| description | text |
| price_per_night | decimal(10,2) |
| capacity | int |
| floor | int |
| size_sqft | int |
| bed_type | enum |
| status | enum |
| is_featured | boolean |
| thumbnail | varchar(255) |
| created_at | datetime |
| updated_at | datetime |

**Table 4.6.3: Physical Database Design for Room Categories**

## Room Categories

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| name | varchar(80) |
| description | text |
| created_at | datetime |

**Table 4.6.4: Physical Database Design for Amenities**

## Amenities

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| name | varchar(80) |
| icon | varchar(80) |

**Table 4.6.5: Physical Database Design for Room Amenities**

## Room Amenities

| Column Name | Data Type & Constraints |
|---|---|
| room_id | int (FK → rooms.id) |
| amenity_id | int (FK → amenities.id) |

**Table 4.6.6: Physical Database Design for Room Images**

## Room Images

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| room_id | int (FK → rooms.id) |
| image_path | varchar(255) |
| caption | varchar(150) |
| sort_order | int |

**Table 4.6.7: Physical Database Design for Bookings**

## Bookings

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| booking_ref | varchar(20) |
| user_id | int (FK → users.id) |
| room_id | int (FK → rooms.id) |
| check_in | date |
| check_out | date |
| nights | int |
| adults | int |
| children | int |
| subtotal | decimal(10,2) |
| tax_amount | decimal(10,2) |
| total_amount | decimal(10,2) |
| special_requests | text |
| status | enum |
| checked_in_at | datetime |
| checked_out_at | datetime |
| handled_by | int (FK → users.id) |
| created_at | datetime |
| updated_at | datetime |

**Table 4.6.8: Physical Database Design for Payments**

## Payments

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| booking_id | int (FK → bookings.id) |
| user_id | int (FK → users.id) |
| amount | decimal(10,2) |
| currency | varchar(3) |
| method | enum |
| status | enum |
| stripe_payment_intent_id | varchar(120) |
| stripe_charge_id | varchar(120) |
| stripe_customer_id | varchar(120) |
| proof_image | varchar(255) |
| proof_submitted_at | datetime |
| reviewed_by | int (FK → users.id) |

| | |
|---|---|
| reviewed_at | datetime |
| review_notes | text |
| paid_at | datetime |
| created_at | datetime |
| updated_at | datetime |

**Table 4.6.9: Physical Database Design for Invoices**

## Invoices

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| booking_id | int (FK → bookings.id) |
| invoice_number | varchar(30) |
| issued_at | datetime |
| due_date | date |
| total | decimal(10,2) |
| notes | text |
| pdf_path | varchar(255) |

**Table 4.6.10: Physical Database Design for Reviews**

## Reviews

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| room_id | int (FK → rooms.id) |
| user_id | int (FK → users.id) |
| rating | int |
| title | varchar(150) |
| body | text |
| is_approved | boolean |
| created_at | datetime |

**Table 4.6.11: Physical Database Design for Messages**

## Messages

| Column Name | Data Type & Constraints |
|---|---|
| id | int (PK) |
| sender_id | int (FK → users.id) |
| recipient_id | int (FK → users.id) |
| subject | varchar(180) |
| body | text |
| is_read | boolean |
| read_at | datetime |
| created_at | datetime |
| updated_at | datetime |

**Table 4.6.12: Physical Database Design for Contact Messages**

## Contact Messages

| Column Name | Data Type & Constraints |
|---|---|
| Id<br>user_id | int (PK)<br>int (FK → users.id) |
| name | varchar(120) |
| email | varchar(191) |
| phone | varchar(30) |
| subject | varchar(200) |
| message | text |
| is_read | boolean |
| created_at | datetime |

**Table 4.6.13: Physical Database Design for Gallery**

## Gallery

| Column Name | Data Type & Constraints |
|---|---|
| Id<br>user_id | int (PK)<br>int (FK → users.id) |
| title | varchar(150) |
| image_path | varchar(255) |
| category | varchar(60) |

| | |
|---|---|
| sort_order | int |
| created_at | datetime |

**Table 4.6.14: Physical Database Design for Sessions**

## Sessions

| Column Name | Data Type & Constraints |
|---|---|
| id | varchar(128) (PK) |
| user_id | int (FK → users.id) |
| payload | text |
| last_activity | int |

**Table 4.6.15: Physical Database Design for Settings**

## Settings

| Column Name | Data Type & Constraints |
|---|---|
| Key<br>user_id | varchar(100) (PK)<br>int (FK → users.id) |
| value | text |

# Chapter 5

# Implementation

## 5.1 Introduction

This chapter presents the implementation of the Sapphire Stay system. It describes how the main system functionalities are implemented, including user authentication, room availability, room booking, payment processing, booking management, and room management. The pseudocode of the major system processes is also presented to illustrate their logical flow before implementation.

## 5.2 Pseudo Code

The pseudocode describes the main processes and logical flow of the Sapphire Stay system. It provides a simplified representation of how users, rooms, bookings, payments, and hotel management operations are handled.

### 5.2.1. User Registration and Login

```
BEGIN
INPUT user details
IF registering THEN
   Validate user information
   Create customer account
   Store user details
ELSE
   INPUT email and password
   Verify login credentials
   IF credentials are valid THEN
      Create user session
   ELSE
      Display login error
   END IF
END IF
```

END

### 5.2.2. Room Search and Availability

BEGIN
INPUT check-in date, check-out date, adults, children
Retrieve available rooms
FOR each room
  Check existing bookings for selected dates
  IF booking dates overlap THEN
    Mark room as unavailable
  ELSE
    Display room as available
  END IF
END FOR
DISPLAY available rooms
END

### 5.2.3. Room Booking

BEGIN
INPUT user, room, check-in, check-out, guests
Check room availability
IF room is available THEN
  Calculate number of nights
  Calculate subtotal, tax, and total
  Generate booking reference
  Create booking
  Set booking status to "pending"
  Display booking details
ELSE
  Display "Room not available"
END IF

END

**5.2.4. Payment Processing**

```
BEGIN
INPUT booking and payment method
Create payment record
Set payment status to "pending"
IF payment method is Stripe THEN
   Process online payment
   IF payment successful THEN
      Set payment status to "completed"
      Confirm booking
   ELSE
      Set payment status to "failed"
   END IF
ELSE
   Submit payment proof
   Keep payment pending for staff/admin verification
END IF
END
```

**5.2.5. Booking Management**

```
BEGIN
INPUT booking and action
IF action = "confirm" THEN
   Set booking status to "confirmed"
ELSE IF action = "check-in" THEN
   Set booking status to "checked_in"
   Record check-in time
ELSE IF action = "check-out" THEN
   Set booking status to "checked_out"
```

Record check-out time

ELSE IF action = "cancel" THEN
Set booking status to "cancelled"
END IF
Save changes
END

**5.2.6. Room Management**

BEGIN
INPUT staff/admin action
IF action = "add" THEN
Enter room details
Save room
ELSE IF action = "update" THEN
Select room
Update room details
ELSE IF action = "delete" THEN
Select room
Delete room
ELSE IF action = "status" THEN
Update room status
END IF
Save changes
END

# Chapter 6

# Testing

## 6.1 Introduction

This chapter presents the testing of the Sapphire Stay system to ensure that its main functionalities work correctly and meet the project requirements. Both white-box testing and black-box testing are used to evaluate the system, identify errors, and verify the correctness and reliability of its functions.

## 6.2 Testing

Software testing is a critical process in the software development life cycle that involves systematically evaluating a software application to identify defects and ensure that it meets the specified requirements. The main purpose of testing is to verify that the system functions correctly, produces expected results, and is reliable for end users. Testing helps improve software quality and ensures that the final product is free from major errors.

### 6.2.1 Black Box Testing

Black box testing is a software testing technique in which the internal structure or code of the system is not considered. Instead, the system is tested based on its inputs and outputs. Test cases are designed according to system requirements, and the output is verified against expected results. This method focuses on validating system behaviour from the user's perspective.

### 6.2.2 White Box Testing

White box testing is a software testing technique in which the internal structure, logic, and code of the system are examined. Unlike black box testing, it focuses on the internal working of the application rather than only inputs and outputs. Test cases are designed based on program logic, control flow, and code paths to ensure that all statements and functions are working correctly.

## 6.3 Black Box Testing

In black box testing, we evaluate the system by providing different inputs and checking whether the outputs match the expected results. The internal code implementation is not considered. We verifying system functionality and user requirements.

### Table 6.3.1: Test Case for User Registration

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|
| Valid Details | Ali Khan<br>alikhan@gmail.com<br>Password: 46721244 | User account created successfully | Account created successfully | Operation Successful |
| Existing Email | Email already registered | Error message displayed | Error message displayed | Operation Successful |

**Table 6.1:** Test Case for User Registration

#### 6.3.2 Test Case 2: Login

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|
| | | | | |

| | | | | |
|---|---|---|---|---|
| Valid Credentials | alikhan@gmail.com Password: 46721244 | User redirected to dashboard | User redirected successfully | Operation Successful |
| Invalid Credentials | Incorrect email or password | Error message displayed | Error message displayed | Operation Successful |

**Table 6.2:** Test Case for Login

### 6.3.3 Test Case 3: Logout

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|
| Logout | Click Logout button | User session terminated and redirected to login page | User logged out successfully | Operation Successful |

**Table 6.3:** Test Case for Logout

### 6.3.4 Test Case 4: View Rooms

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| View Available Rooms | Open Rooms page | Display available rooms with details | Rooms displayed correctly | Operation Successful |
| No Rooms Available | Filter unavailable dates | No rooms available message displayed | Message displayed correctly | Operation Successful |

**Table 6.4:** Test Case for Room Listing

### 6.3.5 Test Case 5: Room Booking

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|
| Valid Booking | Select room and valid dates | Booking created successfully | Booking created successfully | Operation Successful |
| Invalid Dates | Check-out date before check-in date | Validation error displayed | Error displayed correctly | Operation Successful |

**Table 6.5:** Test Case for Room Booking

### 6.3.6 Test Case 6: Payment Processing

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|
| Successful Payment | Valid payment details | Payment completed and booking confirmed | Payment completed successfully | Operation Successful |
| Failed Payment | Invalid payment details | Payment failure message displayed | Message displayed correctly | Operation Successful |

**Table 6.6:** Test Case for Payment Processing

### 6.3.7 Test Case 7: Booking Management

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|
| Confirm Booking | Staff changes booking status to Confirmed | Booking status updated | Status updated successfully | Operation Successful |
| Cancel Booking | Staff changes booking status to Cancelled | Booking status updated | Status updated successfully | Operation Successful |

**Table 6.7:** Test Case for Booking Management

### 6.3.8 Test Case 8: Room Management

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|
| Add New Room | Enter room details | Room added successfully | Room added successfully | Operation Successful |
| Missing Required Data | Leave required fields empty | Validation error displayed | Error displayed correctly | Operation Successful |

**Table 6.8:** Test Case for Room Management

### 6.3.9 Test Case 9: Contact Message

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|
| Valid Message | Name, Email and Message | Message submitted successfully | Message submitted successfully | Operation Successful |
| Empty Fields | Missing required fields | Validation error displayed | Error displayed correctly | Operation Successful |

**Table 6.9:** Test Case for Contact Messages

### 6.3.10 Test Case 10: Invoice Generation

| Case | Actual Input | Expected Output | Actual Output | Remarks |
|---|---|---|---|---|
| Completed Booking | Generate invoice | Invoice generated successfully | Invoice generated successfully | Operation Successful |
| Invalid Booking ID | Generate invoice for non-existing booking | Error message displayed | Error displayed correctly | Operation Successful |

**Table 6.10:** Test Case for Invoice Generation

# Chapter 7

# Conclusion and Future Enhancement

## 7.1 Introduction

Sapphire Stay aims to digitalize and improve the traditional manual hotel management system by developing a centralized web-based platform. This system is designed specifically for a single hotel to manage room bookings, customer records, and staff operations efficiently. It replaces manual registers with a structured digital solution that improves accuracy, reduces errors, and enhances overall hotel operations.

## 7.2 Result and Discussion

**Room Management System**: Successfully allows admin to add, update, and manage room details including availability and pricing.

**Booking System:** Customers can view available rooms and make bookings efficiently through the system.

**User Management:** Separate dashboards for Admin, Staff, and Customers ensure proper role-based access and control.

**Booking Management:** Staff can view, update, and manage booking status in real time.
Database Integration: MySQL database integration ensures secure storage and smooth retrieval of hotel data.

**Secure Authentication System:** Login and registration system ensures only authorized users can access the platform.

## 7.3 Future Enhancements

Although Sapphire Stay fulfills the current requirements of a single-hotel management system, there is still scope for future improvement and expansion.

**Mobile Application:** A dedicated mobile application can be developed to provide easier access for customers and staff, allowing room booking, availability checks, and management tasks directly from smartphones.

**Multi-Hotel Support:** The system can be extended to support multiple hotels under a single platform, enabling centralized management of different hotel branches and their operations.

**Dynamic Pricing System:** A dynamic pricing feature can be introduced where room prices automatically adjust based on demand, season, weekends, or occupancy levels. This would help optimize revenue management for the hotel.

**Self Check-in / QR Code System:** A QR-based check-in system can be added where customers receive a QR code after booking, which can be scanned at the reception for faster and contactless check-in.

## 7.4 Summary

This chapter presented the conclusion and future scope of the Sapphire Stay system. The project successfully achieved its goal of replacing manual hotel management processes with a structured digital system. The system provides efficient room booking, user management, and staff operations in a simple and user-friendly interface. Future improvements can further enhance the system by introducing mobile support, payment integration, and real-time features, making it more scalable and modern.

# Reference and Bibliography

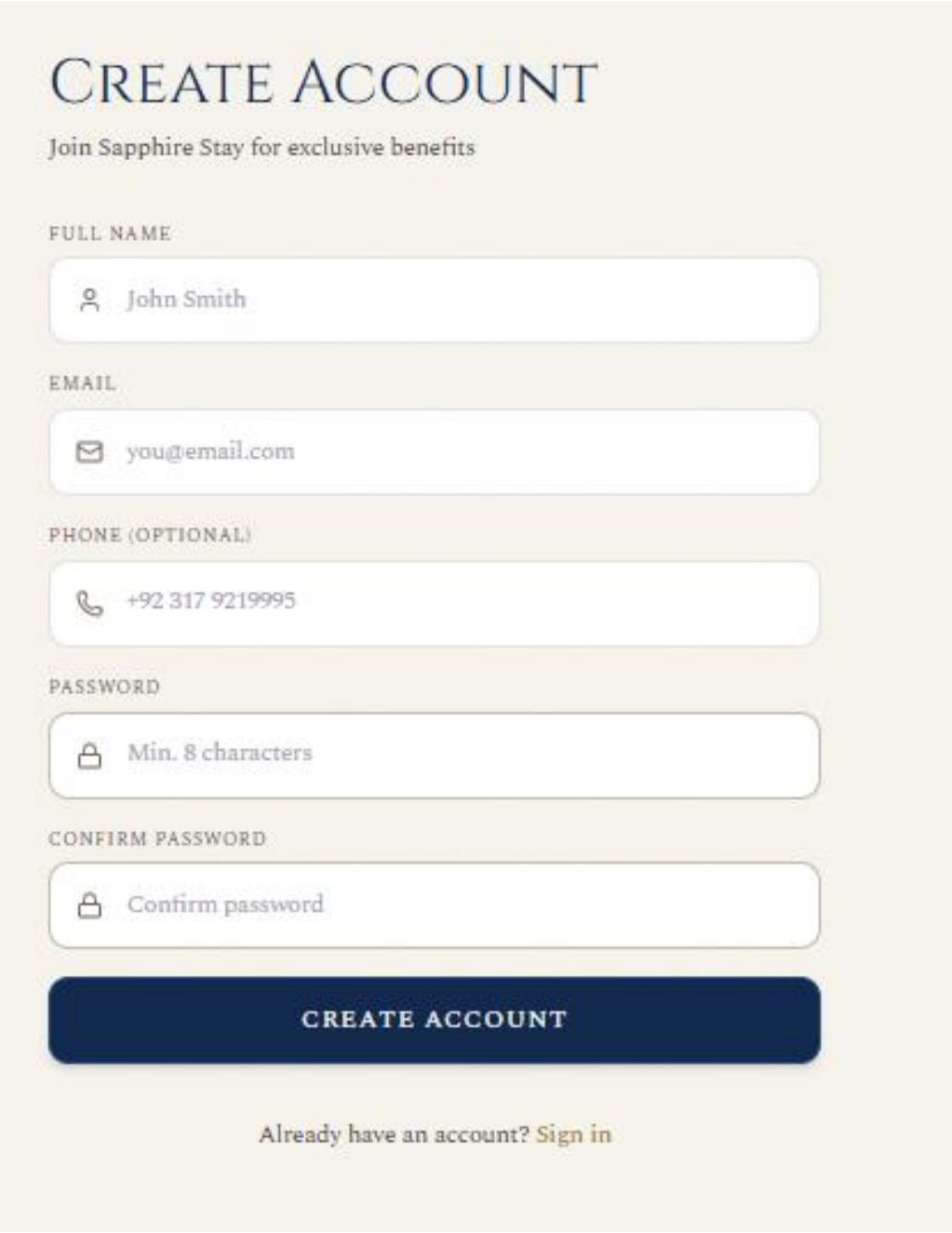

1. PHP Documentation. https://www.php.net/manual/en/
2. Oracle. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
3. MDN Web Docs. Web Development Documentation: HTML, CSS and JavaScript. Mozilla. https://developer.mozilla.org/en-US/
4. Tailwind CSS.: https://tailwindcss.com/docs
5. Apache Friends. XAMPP. https://www.apachefriends.org/
6. OWASP Foundation. SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
7. OWASP Foundation. Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
8. Stripe. Stripe Payments Documentation. https://docs.stripe.com/payments

# Appendix:

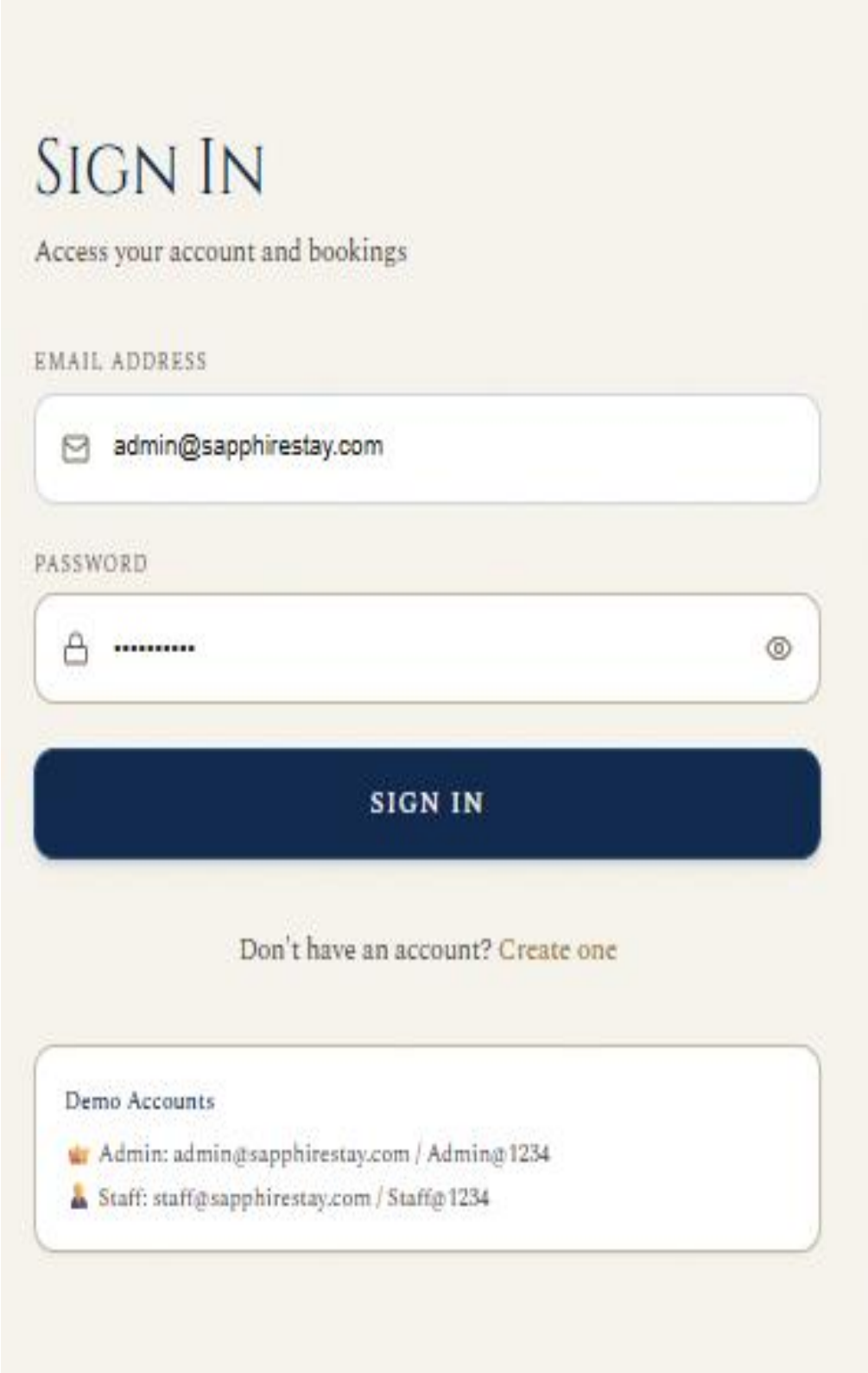
## Home Page



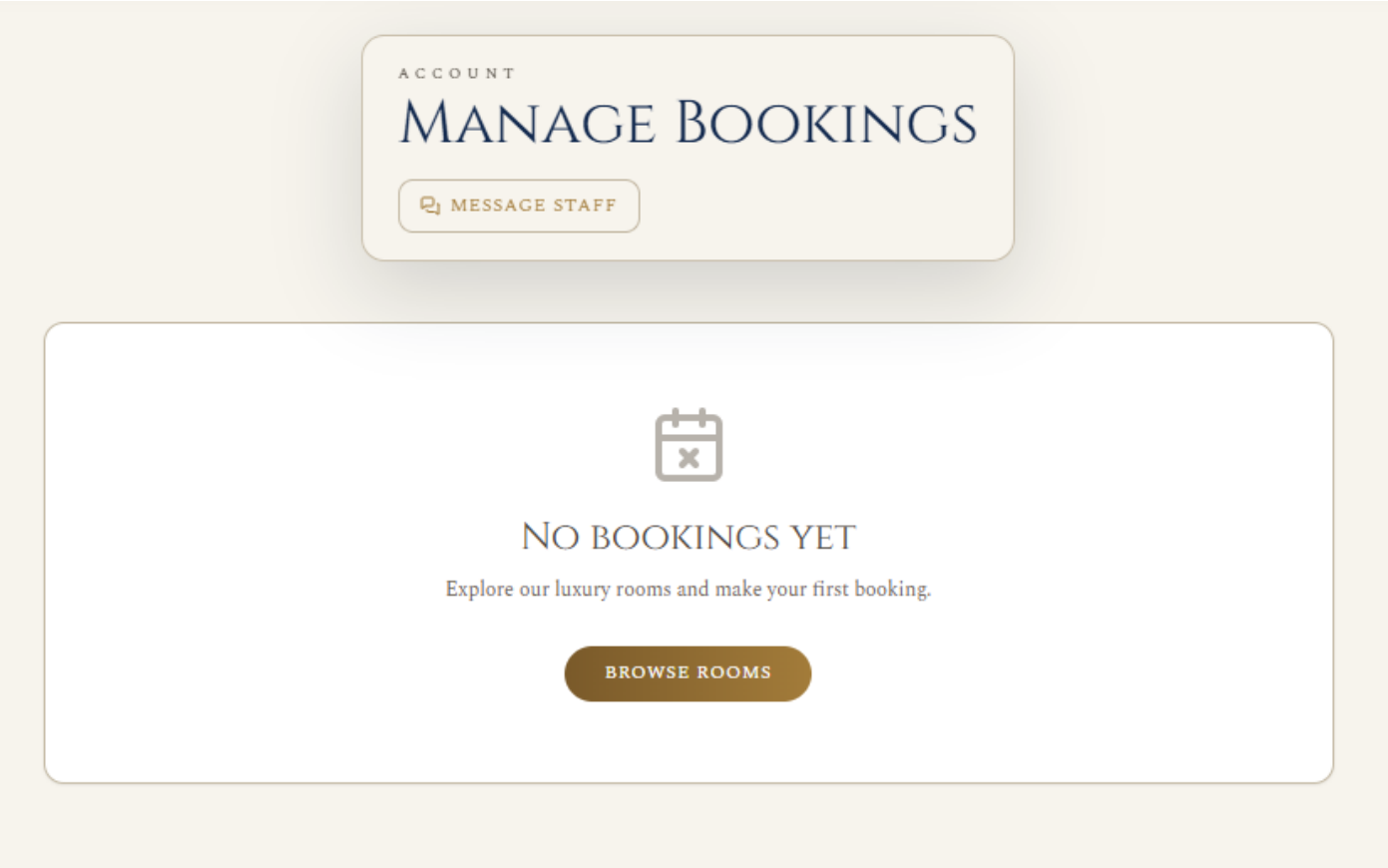
**User Registration Page**



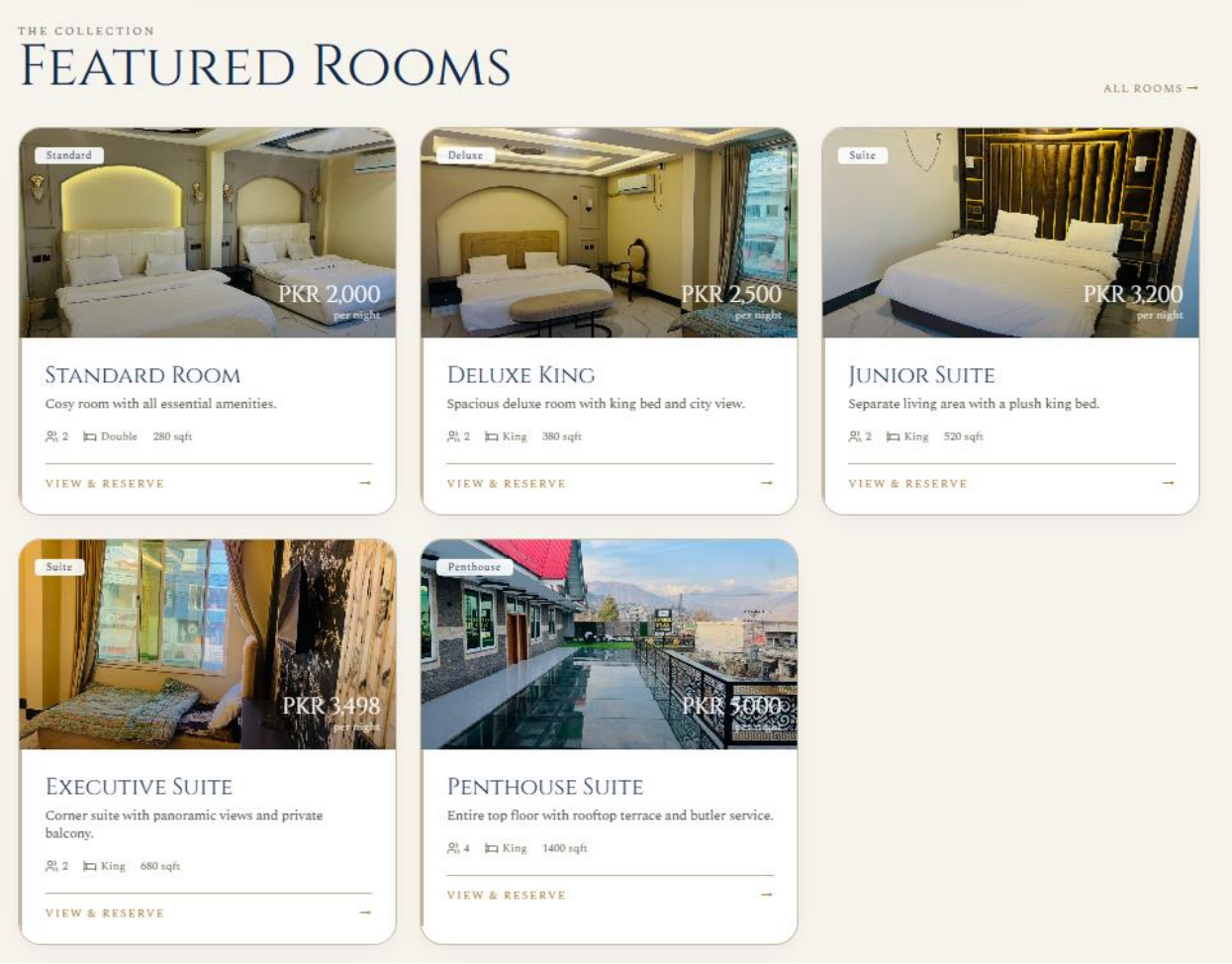
**Sign In Page**


SIGN IN
Access your account and bookings
EMAIL ADDRESS
admin@sapphirestay.com
PASSWORD
SIGN IN
Don't have an account? Create one
Demo Accounts
Admin: admin@sapphirestay.com / Admin@1234
Staff: staff@sapphirestay.com / Staff@1234

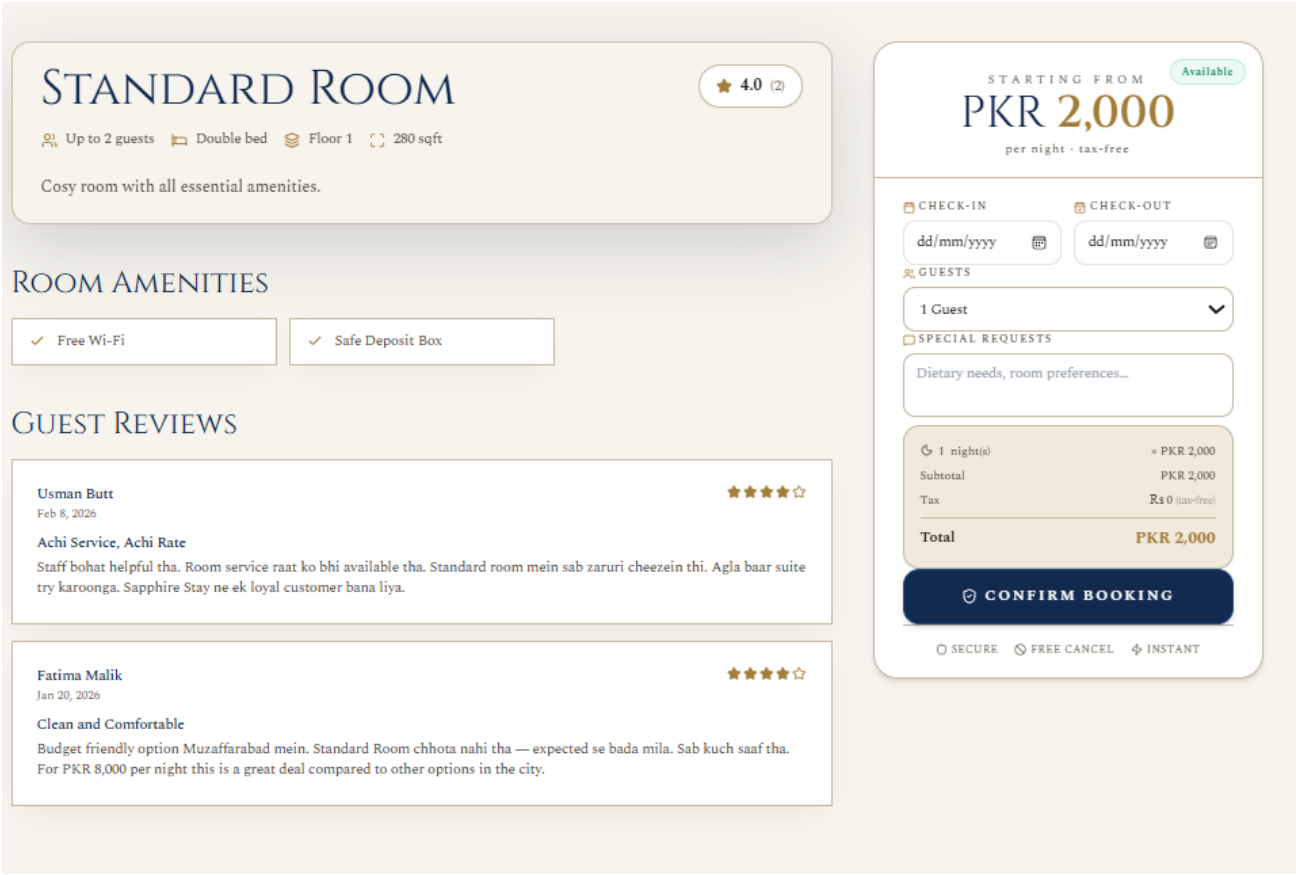
**Customer Dashboard**



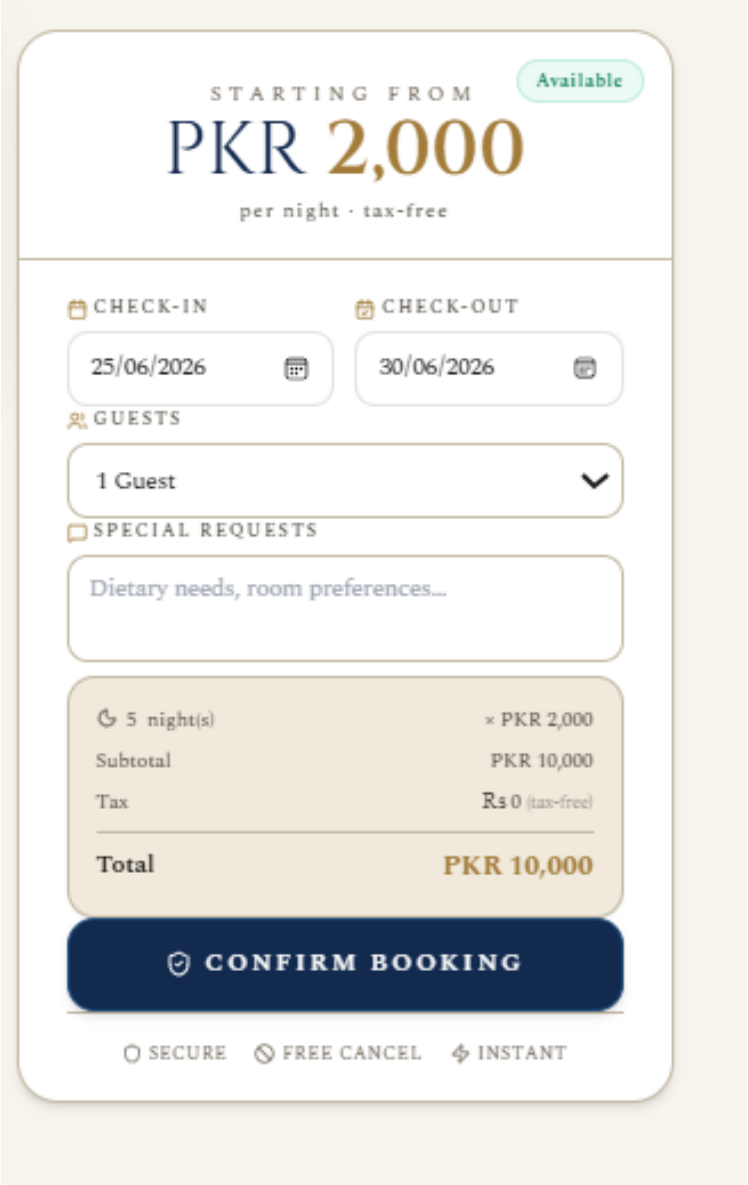
## Room Listing Page



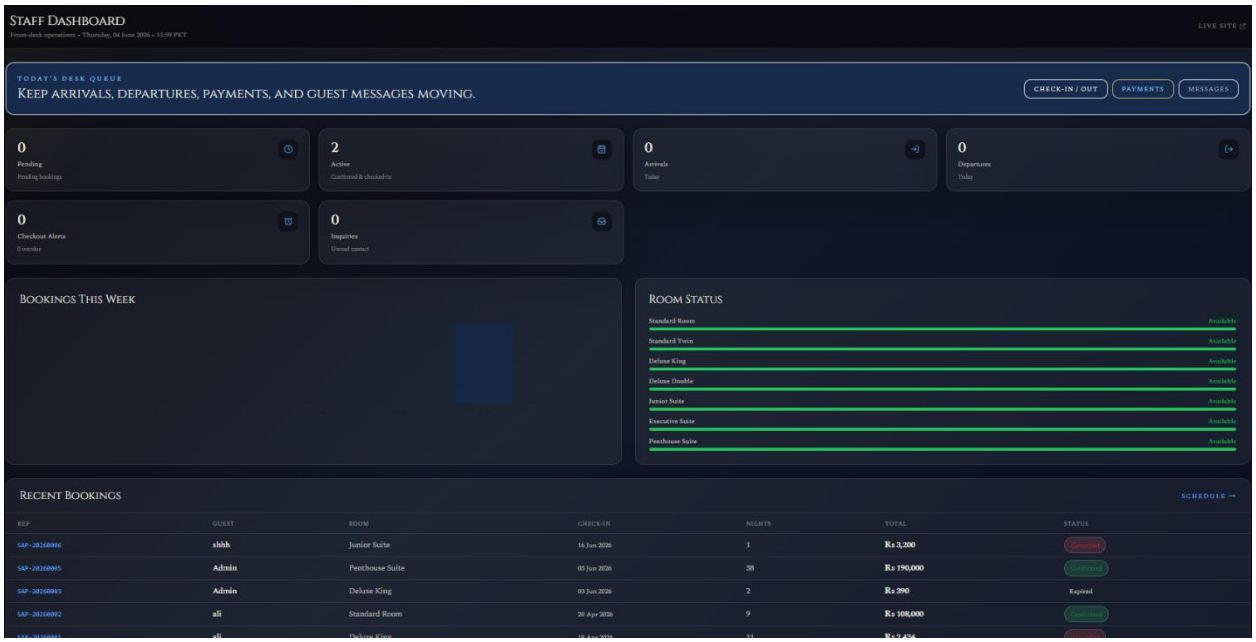
## Room Details Page



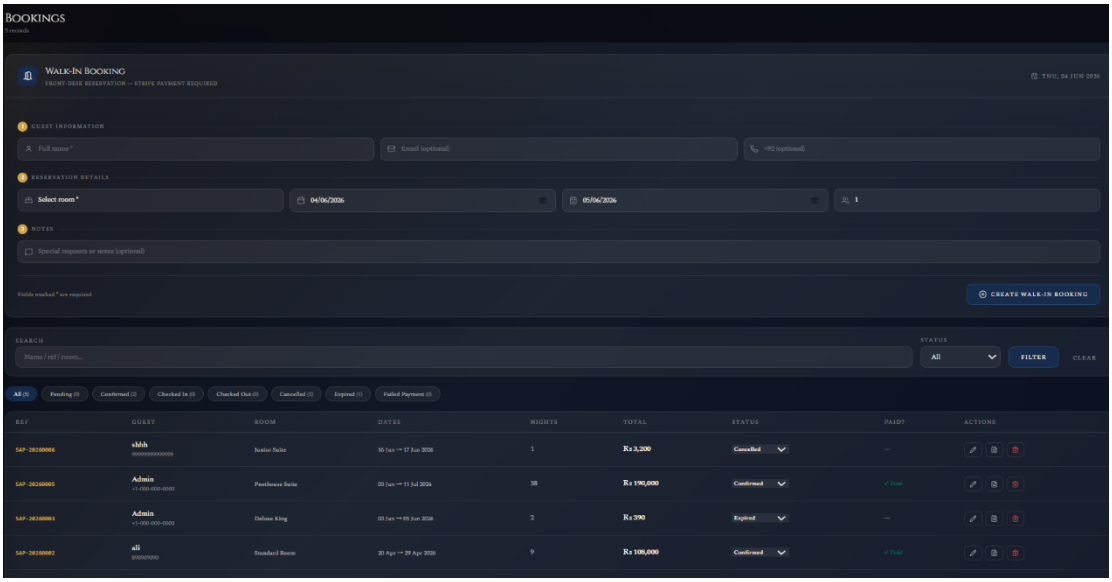
## Booking Page



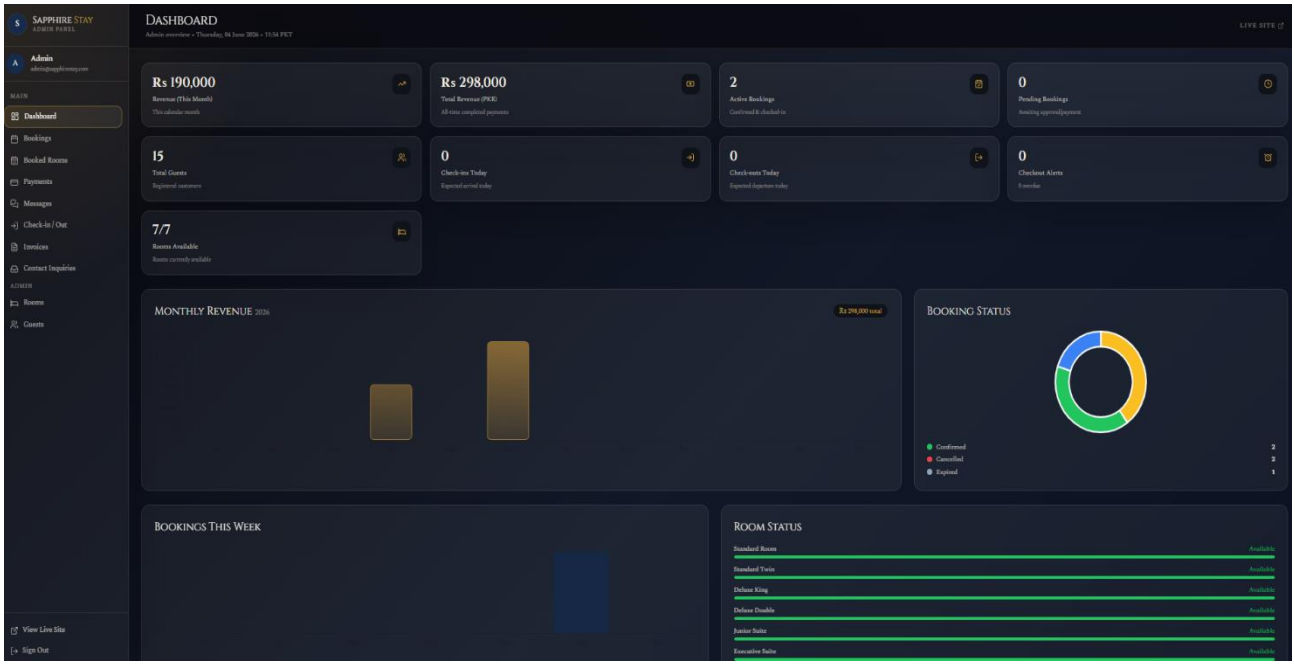
## Staff Dashboard




## Booking Management

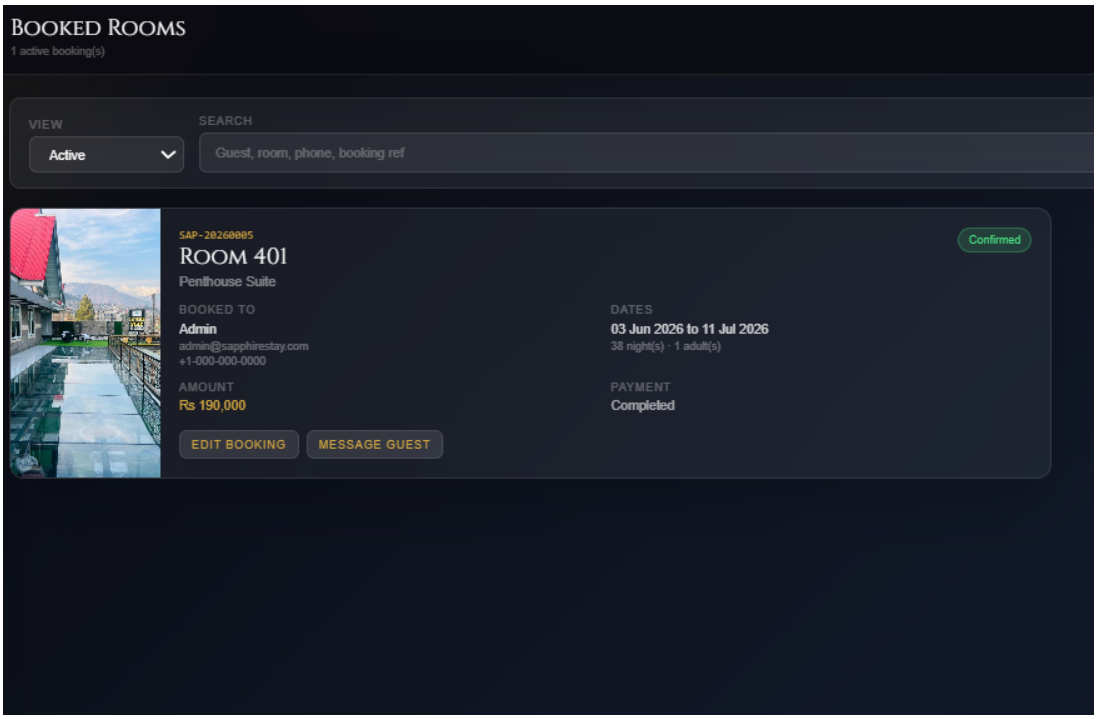


## Admin Dashboard

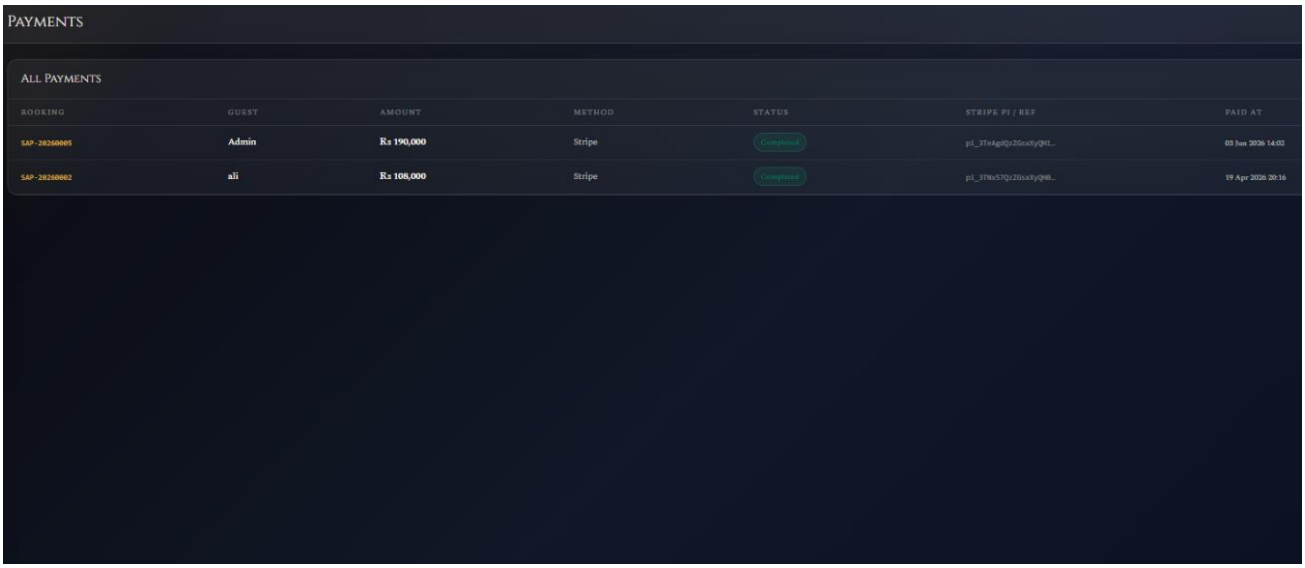



## Room Management

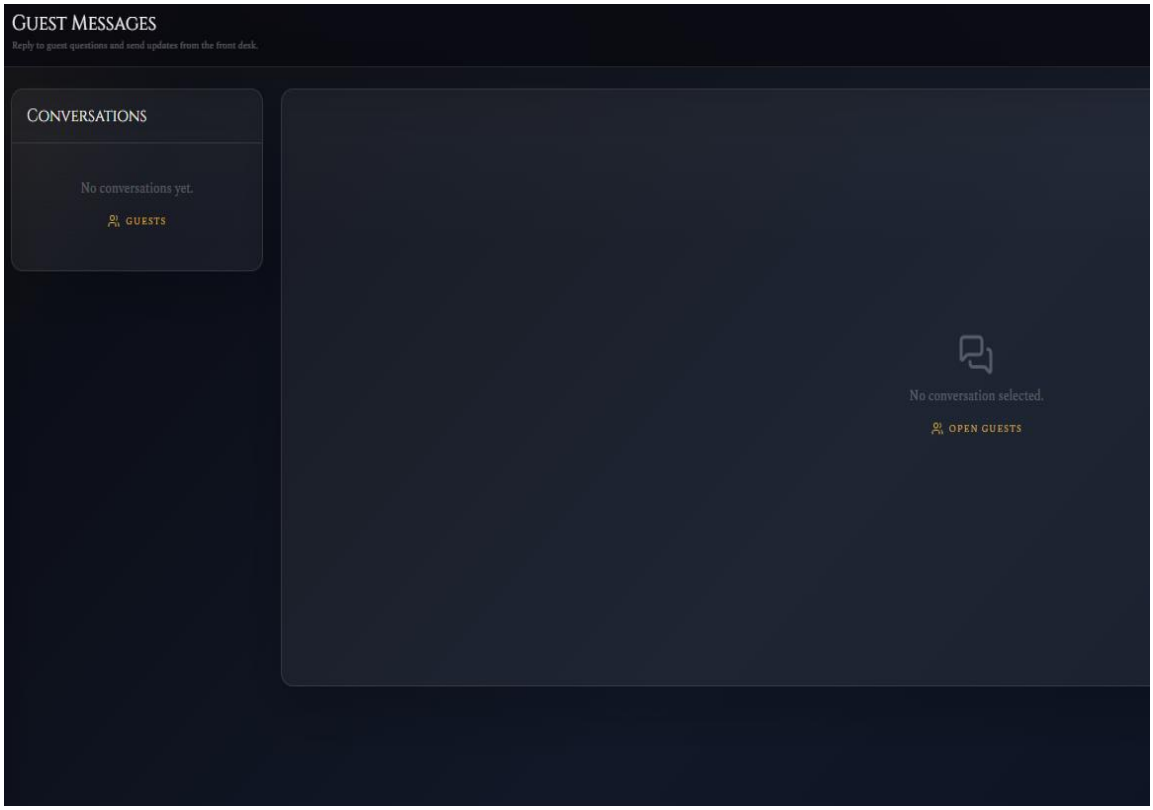
.



## Payment Management




## Contact Messages Management



## Invoice Generation

